RelaLeap Tiny Shakespeare Residual-Layer Work So Far

Date: 2026-07-09
Audience: external human/coding-agent collaborator
Prepared by: ZeroBot / OpenClaw research-agent
Project: RelaLeap - residual-layer learning prototypes for small transformers

Executive summary

RelaLeap has produced a substantial experimental harness and many small-scale residual-layer experiments around Tiny Shakespeare-style character/token language modeling, but the scientific state is deliberately conservative.

The current evidence says:

1. The harness works. We can train residual parameters on a frozen tiny transformer, preserve base-model invariants, run local/Colab-compatible command-line experiments, emit artifact contracts, and compare runs reproducibly.
2. Simple residual columns can improve small language-model losses. Early Tiny Shakespeare char-level experiments showed small but real CE reductions from residual training, and a label-free HEP/settling alpha could improve loss within a logit-drift budget.
3. Some engineering choices were promoted as defaults. Top-k=2 support width and a contextual MLP support router were promoted as empirical defaults under local/Colab evidence.
4. The stronger "causal columns" claim did not pass. Later audits found that apparent top-k=2 cooperation was confounded by active rank/residual scale/support frequency, and broad top-k=2 causal-cooperation claims were locally closed.
5. PC-style residual-objective variants were tried but not promoted. A simple logit-MSE/anchored PC residual objective did not beat supervised CE residual training under the checked gates.
6. ACSR/routing innovation alone was demoted. Anticipatory contextual support routing produced diagnostic signal but did not clear dense/control/retention-churn gates.
7. The project then pivoted to SLT-guided residual factorization. That work produced a local synthetic/SLT validation scaffold in OpenClaw, but it has not yet produced validated Tiny Shakespeare SLT evidence.
8. As of 2026-07-09, Ben changed sequencing again. The next first track is HDPC/ePC homotopy distillation plus PC crowns on Tiny Shakespeare, with SLT inputs and columnar residual models treated as later upgrades.

The practical takeaway for outside coding agents: start from the public GitHub branch agent/slt-pregate-v2-cache to understand the earlier Tiny Shakespeare residual-layer harness and reports, but do not treat earlier columnar/SLT claims as validated science. The next clean implementation path is a bounded HDPC/ePC branch, with tests before trainers.

Main repository references

Public GitHub repository:

- https://github.com/bgoertzel-sing/relaleap

Important branches observed on GitHub:

- main at af35d62371c6d9767ee3ab886526325e38cc838f
- agent/slt-pregate-v2-cache at 5b959e331f73ad0fa0004bed60c3327844272de8

The branch agent/slt-pregate-v2-cache is the richer branch for historical Tiny Shakespeare residual-layer work. It contains many configs, experiments, docs, and checked result summaries. The main branch is not enough for a full handoff.

Useful URLs:

- Repo root: https://github.com/bgoertzel-sing/relaleap
- Rich branch: https://github.com/bgoertzel-sing/relaleap/tree/agent/slt-pregate-v2-cache
- README: https://github.com/bgoertzel-sing/relaleap/blob/agent/slt-pregate-v2-cache/README.md
- Phase 0 report: https://github.com/bgoertzel-sing/relaleap/blob/agent/slt-pregate-v2-cache/docs/phase0_report.md
- 2026-06-23 research pivot: https://github.com/bgoertzel-sing/relaleap/blob/agent/slt-pregate-v2-cache/docs/research_pivot_2026_06_23.md
- Automation status / chronological run log: https://github.com/bgoertzel-sing/relaleap/blob/agent/slt-pregate-v2-cache/AUTOMATION_STATUS.md

Current OpenClaw local planning/SLT-validation repository:

- Local path: projects/relaleap/repos/relaleap
- Branch: agent/train-time-causal-slice1
- Commit: 622ede2 (Fix bootstrap_ci import: move to null_reports module; add to __all__)
- Current tests: 93 passed from python3 -m pytest tests -q

New HDPC/ePC worktree prepared on 2026-07-09:

- Local path: projects/relaleap/worktrees/tinyshakespeare-hdpc
- Branch: agent/tinyshakespeare-hdpc
- Base commit: 622ede2
- No Runpod provisioning yet.

Historical arc

1. Initial RelaLeap mission

The original public README describes RelaLeap as an experimental codebase for residual-layer learning prototypes, beginning with columnar PC/CC residual layers on frozen backprop-trained transformer bases.

Initial goal:

> Validate a minimal experimental harness on char-level Tiny Shakespeare before moving to larger GPU experiments.

The first campaign was intentionally tiny:

- 2-layer char-level base model
- hidden dimension 32
- sequence length 32
- frozen base
- one residual insertion site
- 8 residual columns
- 4 atoms per column

- top-k=1 routing
- residual training only, not frontier-scale training

Early guardrails required:

- zero-initialized columns preserve base logits
- frozen base parameters do not change during residual-layer training
- HEP alpha 0 matches ordinary inference
- each run emits summary.json, metrics.csv, and notes.md

Primary source: README.md and docs/phase0_report.md on agent/slt-pregate-v2-cache.

2. Phase 0 Tiny Shakespeare char-level harness

Phase 0 validated infrastructure, not a major scientific claim.

Checked configs:

- configs/char_smoke.yaml: supervised CE residual update
- configs/char_smoke_pc.yaml: PC-style logit-MSE residual update
- configs/char_smoke_hep.yaml: supervised CE residual update with HEP alpha sweep 0.0, 0.25, 0.5, 1.0

Phase 0 setup:

- Tiny Shakespeare char-level data
- 2-layer hidden-dim-32 base
- sequence length 32
- one insertion site
- 8 residual columns
- 4 atoms per column
- top-k=1
- seed 1
- 10 residual training steps

Reported evidence:

- Phase 0 model invariants: 12/12 passing
- artifact invariants: 9/9 passing
- baseline comparison against checked local baseline: passing with zero mismatches
- Colab/GPU artifact tree had been checked by the artifact checker

Loss summary from docs/phase0_report.md:

Experiment	Objective	Initial residual loss	Final residual loss	Delta
char_smoke	supervised CE	3.61089253	3.56317067	-0.04772186
char_smoke_pc	PC logit MSE	0.02878798	0.02869168	-0.00009630
char_smoke_hep	supervised CE	3.61089253	3.56317067	-0.04772186

HEP alpha sweep:

Alpha	Loss	Max logit delta	Gate
0.0	3.5631706715	0.0	baseline
0.25	3.5519564152	0.0518576503	accepted
0.5	3.5410408974	0.1037149131	rejected: over 0.1 logit-delta budget
1.0	3.5201268196	0.2074296474	rejected: over 0.1 logit-delta budget

Interpretation: the harness was validated and HEP alpha 0.25 was acceptable under a conservative logit-drift budget, but this was still only a small infrastructure-scale result.

3. Research pivot: CE became a guardrail, not the main claim

On 2026-06-23 the project pivoted from tiny CE/perplexity improvements toward causal residual-column evidence.

The new central question became:

> Can the residual layer learn causally separable, reusable corrections with less interference than matched alternatives?

This changed the evaluation target. CE/perplexity remained guardrails, but promotion required evidence such as:

- support-width deconfounding
- oracle-support regret
- functional churn, not just support identity churn
- causal intervention fingerprints
- continual-learning retention
- finite-update commutators
- dense-teacher residual distillation
- matched dense/rank/norm/null controls

Primary source: docs/research_pivot_2026_06_23.md.

4. Support-width and router results

Two important empirical defaults were promoted before later causal caveats narrowed the claims.

Top-k=2 support width

Decision report:

- results/reports/residual_support_width_promotion_gate_satisfaction/decision_report.json

Reported decision:

- decision: satisfy_residual_support_width_promotion_gate
- promotion_gate_satisfied: true
- promote_support_width_default: true
- selected default support width: top_k = 2

Rationale quoted in the decision report: local and Colab artifact-backed evidence showed top-k=2 improving ordinary alpha-0 supervised CE loss and final residual loss over top-k=1 at larger-char and tokenized scales while holding supervised CE and temporal-clipped HEP fixed.

Important caveat: this is an empirical support-width default, not proof of causal column cooperation.

Contextual MLP support router

Decision report:

- results/reports/contextual_support_router_promotion_gate_satisfaction/decision_report.
json

Reported decision:

- decision: satisfy_contextual_support_router_promotion_or_repeat_gate
- promote_contextual_support_router_default: true
- default support width remained top-k=2
- default objective remained supervised CE
- default support-stress mitigation remained temporal-clipped HEP

Rationale: matching local and real-Chrome Colab evidence showed the contextual MLP support router lowered alpha-0 CE loss and expanded support utilization versus the linear top-k=2 router.

Caveat: nonzero HEP alphas were not the driver; the default change was scoped to support routing.

5. PC-style residual objectives: tried, then stopped

The branch did contain PC-flavored residual objective work, but it was not the HDPC/ePC method now being planned.

Relevant report:

- results/reports/anchored_pc_residual_objective_decision/decision_report.json

Reported decision:

- decision: stop_pc_residual_objective_validation
- continue_pc_residual_objective_validation: false
- promote_residual_learning_method: false
- default residual objective remained supervised CE

Rationale: the CE-anchored PC objective closed much of the unanchored PC supervised-CE HEP loss gap, but still did not beat supervised CE residual training in the checked local and Colab artifacts.

Interpretation: the old PC residual objective was a shallow/logit-level proxy. It should not be confused with the new HDPC/ePC homotopy-distillation plan, which is a deeper predictive-coding conversion/distillation and crown-learning program.

6. Temporal clipped HEP / support-stress mitigation

Relevant report:

- results/reports/temporal_clipped_hep_token_larger_colab_decision/decision_report.json

Reported decision:

- decision: select_temporal_label_free_support_stress_candidate
- deployable_label_free_signal: true
- selected label-free support-stress candidate: temporal consistency
- promote_to_default_support_stress_mitigation: false

Rationale: temporal clipped HEP was deployable at inference time and produced a nonzero alpha with support-stress loss improvement inside stability budgets, while entropy did not improve loss and the guided oracle was diagnostic-only.

Interpretation: useful candidate mechanism, not a promoted scientific claim.

7. Causal audits narrowed the claims

Later audits closed the broad top-k=2 causal-cooperation claim.

Relevant reports:

- results/reports/token_larger_topk2_causal_cooperation_stop_decision/decision_report.json
- results/reports/token_larger_active_rank_matched_topk1_causal_bracket_audit/decision_report.json

Top-k=2 stop decision:

- decision: stop_topk2_causal_cooperation_claim
- topk2_causal_cooperation_claim_supported: false
- topk2_causal_cooperation_claim_closed_locally: true
- colab_topk2_replication_warranted: false

Rationale: visible top-k=2 pair synergy survived only a weak sign-flip null. Best-swap selection control was negative; top-k=2 missed fixed-support and functional-churn cleanliness gates; support-frequency controls were unidentified rather than supportive.

Rank-matched top-k=1 bracket decision:

- decision: confirm_active_rank_matched_topk1_causal_bracket
- rank_matched_topk1_primary_causal_bracket: true
- top-k=2 remained a reference condition only
- support-frequency percentile claim unsupported

Interpretation: top-k=2 remains a useful CE/support-routing default, but causal separability should be audited under active-rank-matched top-k=1 or similarly deconfounded brackets.

8. ACSR/routing-only direction was demoted

Anticipatory Contextual Support Routing (ACSR) was explored after Ben’s 2026-06-27 and 2026-06-30 directions. It was valuable diagnostically but failed promotion.

Relevant report:

- results/reports/acsr_negative_evidence_closeout/summary.json

Reported status:

- decision: acsr_negative_evidence_closeout_branch_selected
- claim_status: acsr_promotion_path_demoted_to_diagnostic_no_default_change
- requires_gpu_now: false
- selected next action: demote ACSR to diagnostic status

Rationale: ACSR beat simple nulls but failed parameter-matched and retention-churn guardrails; dense-teacher, rank/norm, MLP, norm-budgeted, commutator, and continual-learning-repeat evidence did not establish a sparse-specific mechanism.

9. Dense-teacher and sparse-coding feasibility

The project then probed whether residuals from a dense teacher could be represented by sparse/oracle structures and whether deployable routers could imitate them.

The automation status records a key local result:

- dense teacher improved base CE from 1.592115 to 1.387565
- oracle top-k orthogonal sparse coding was strong locally: CE 1.342901, R2 0.839040
- same-router flat control also strong: CE 1.399378, R2 0.689996
- deployable learned router/scalar coding reached only CE 1.531542, R2 0.217441, failing the low oracle-gain-regret gate

Interpretation: oracle sparse columnability existed in a local assay, but deployable sparse routing/value learning was the blocker, and flat controls remained strong. No GPU or scientific promotion followed from this.

10. SLT residual-layer mandate, v0/v2 closeouts, and local OpenClaw scaffold

On 2026-07-02 Ben supplied the "SLT and Residual Layers" direction. The core shift was to stop promoting residual bases because they reconstruct well, and instead promote only if SLT/free-energy/causal-factorization evidence passes strict gates.

10.1 Posthoc seven-arm SLT pregate: useful but closed

The first SLT branch was a posthoc/frozen-cache residual-basis pregate. It compared seven interpreted arms:

- flat control
- SVD/low-rank
- orthogonal sparse diagnostic
- nonorthogonal learned dictionary
- CE-gradient-aligned dictionary
- Fisher/Gauss-Newton-aligned dictionary
- rank-one atom columns

GitHub closeout source:

- docs/slt_posthoc_branch_closeout_for_ben.md on branch agent/slt-pregate-v2-cache

Closeout result:

- 7 interpreted arms
- 0 arms beating all required null controls
- 4 arms beating dense/SVD controls only
- 0 GPU-gate candidates
- advance_to_gpu_validation: false
- promotion_allowed: false

Required nulls that blocked promotion included:

- label_shuffled_teacher
- context_shuffled_support
- misaligned_residual_target

Interpretation: sparse/dictionary arms sometimes looked better than dense/SVD controls, but not better than required null controls. Therefore the branch was useful as negative evidence and a scaffold, not as causal residual-factorization evidence.

10.2 v2 residual-cache/oracle-identifiability pregate: benchmark passed, deployable mechanisms failed

The v2 branch introduced a cleaner ResidualCacheV2 formulation: hidden residual-stream targets in d_model, train/val/test cache splits, and shared base/dense teacher/frozen tail/insertion site/dataset identity.

GitHub closeout source:

- docs/slt_pregate_v2_closeout_for_ben.md on branch agent/slt-pregate-v2-cache

What passed:

- ResidualCacheV2 enforced hidden residual-stream targets rather than class-logit residual substitutes.
- CPU-only oracle identifiability passed across six Tier-A regimes.
- Structured oracle positive controls reached best oracle R2 1.0 for exact_factorized, shared_core_redundant, synergistic_pair, low_rank_trap, and oblique_dictionary.
- random_null did not produce a non-null false positive.

What failed or stayed closed:

- gated shared-core sparse residual
- norm-budgeted separable core/deviation
- whitened orthogonal support-gated pairwise

Interpretation: the benchmark could reveal known structure, but the handcrafted deployable mechanisms did not discover it strongly enough. GPU validation, real-cache work, and WBIC/LLC claims were blocked.

10.3 Train-time causal factor learner and SLT-estimator validation scaffold

The local OpenClaw project records preserve the stricter train-time/SLT direction as:

- projects/relaleap/docs/train_time_causal_factor_preregistration.md
- projects/relaleap/docs/slt_estimation_validation_checklist.md
- projects/relaleap/docs/slt_estimator_validation_plan_summary.md

The train-time causal factor preregistration specifies:

- identity-initialized rank-one residual atoms grouped into columns
- sparse column masks/supports
- exact ablation calibration
- commutator/leakage/support-regret gates
- SLT/WBIC/SGLD finite-sample LLC proxies over actual trainable blocks
- dependency-aware null controls
- fail-closed reporting

The OpenClaw local implementation branch has built synthetic and SLT-estimation infrastructure:

- branch: agent/train-time-causal-slice1
- commit: 622ede2
- modules include synthetic regimes, arms, audits, nulls, reports, SLT contracts, SGLD/WBIC core, analytic calibration registry, MAP/prior sensitivity, RelaLeap-shaped benchmarks, and decision-report gates
- verification run on 2026-07-09: 93 passed from python3 -m pytest tests -q

Important limitation: this is not yet validated Tiny Shakespeare SLT evidence. It is a local CPU scaffold and synthetic/estimator validation infrastructure.

11. Current 2026-07-09 sequencing: HDPC/ePC first

Ben then supplied two predictive-coding papers:

1. HDPC working draft: homotopy distillation from backprop transformers into PC-consistent learners, plus energy-coupled predictive-coding crowns.
2. Goemaere et al. 2026 ePC paper: error-based predictive coding for fast/deep digital simulation.

Ben's 2026-07-09 sequencing decision:

- try HDPC/ePC first on Tiny Shakespeare
- use Runpod compute resources only after bounded approval
- afterwards consider upgrades using:
1. SLT inputs to the process
2. columnar models for the residual layer

This was recorded locally as decision:

- projects/relaleap/DECISIONS.md
- D-20260709-hdpc-before-slt-columnar-upgrades

Prepared worktree:

- projects/relaleap/worktrees/tinyshakespeare-hdpc
- branch agent/tinyshakespeare-hdpc
- base commit 622ede2

No Runpod job has been provisioned yet.

What has actually run on Tiny Shakespeare?

Known completed Tiny-Shakespeare-style work from the public branch:

- Phase 0 char-level smoke harness
- supervised CE residual update
- PC-style logit-MSE residual update
- HEP alpha sweeps
- support-width experiments on larger char/tokenized variants
- contextual support-router promotion gates with local/Colab evidence
- causal audits and stop decisions over token-larger variants

What has not yet run, as of this handoff:

- HDPC/ePC homotopy distillation on Tiny Shakespeare
- PC-consistent transformer base conversion under the HDPC plan
- energy-coupled PC crown experiments under the HDPC plan
- validated SLT/WBIC/LLC estimates on Tiny Shakespeare residual layers
- SLT-guided columnar residual-layer training with passed estimator gates
- Runpod execution for the HDPC plan

Suggested starting points for external coding agents

A. Read first

1. README.md on agent/slt-pregate-v2-cache

2. docs/phase0_report.md
3. docs/research_pivot_2026_06_23.md
4. AUTOMATION_STATUS.md - skim, but do not assume every old branch remains active
5. Local OpenClaw docs if available:
  - projects/relaleap/docs/train_time_causal_factor_preregistration.md
  - projects/relaleap/docs/slt_estimation_validation_checklist.md
  - projects/relaleap/DECISIONS.md

B. Reproduce old harness before modifying it

On the public branch, the README suggests commands such as:

```
python -m relaleap.experiments.compare
python -m relaleap.experiments.check_artifacts \
  --comparison-dir results/comparisons/colab_phase0 \
  --baseline-reference baselines/phase0_char_smoke_comparison.json
```

A coding agent should first verify the checked artifact contracts and only then modify model code.

C. Do not pursue these as first tasks

Avoid starting with:

- another top-k=2 causal-cooperation proof attempt
- another ACSR-only routing variant
- another shallow PC logit-MSE residual objective
- GPU/Runpod scaling before local HDPC/ePC unit tests pass
- SLT-guided claims before SLT estimator calibration passes

These have either been demoted, closed locally, or are explicitly sequenced later.

D. Current recommended first implementation task

Implement the HDPC/ePC Tiny Shakespeare scaffold on agent/tinyshakespeare-hdpc, CPU-first, tests before trainers.

Minimum tests from the HDPC paper appendix:

1. zero-error identity: PC-wrapped model with zero errors matches vanilla forward
2. KD anchor: student=teacher gives near-zero KD loss and PC gradient at T in {1,4,16}
3. energy monotonicity: energy non-increasing over inner ePC steps for stable lambda
4. BP endpoint: T=1 ePC weight gradient matches lambda times BP gradient
5. PC endpoint distinctness: large T gradient separates from BP on perturbed student
6. crown identity: zero-initialized crown changes no logits
7. toy sPC/ePC equivalence on a small MLP

Suggested first model/corpus:

- Tiny Shakespeare char-level corpus
- very small GPT/decoder or existing RelaLeap char transformer
- sequence length 32 or 64 initially
- pure KD self-distillation anchor first
- lambda 0.05, T schedule {1,2,4,8}
- no dropout, no KV cache inside relaxation
- fp32 error tensors

- log PC-vs-BP cosine, energy profiles, held-out perplexity delta, update sparsity

E. Runpod approval packet needed before remote compute

Before starting Runpod, prepare and get explicit approval for:

- provider/account context
- GPU type, count, region, image/template, storage
- expected duration and maximum wall-clock time
- current price source and maximum dollar budget
- data upload scope
- artifact return path
- stop/terminate condition
- what happens if tests fail mid-run

Current autonomous spend budget is zero, so no paid resource should be started without approval.

Known risks and traps

1. Confusing old PC objective work with HDPC/ePC. The old PC logit-MSE/anchored objective failed to beat CE; HDPC/ePC is a different, deeper method.
2. Claiming causal columns too early. Earlier top-k=2 gains were narrowed by deconfounding audits.
3. Overreading SLT proxies. The project explicitly treats WBIC/SGLD estimates as finite-sample proxies requiring calibration, not exact RLCTs.
4. Flat/dense controls are strong. Many sparse mechanisms beat weak nulls but not same-router flat or dense/rank/norm controls.
5. Deployable routing is hard. Oracle sparse coding may look good while learned deployable routers fail to capture oracle gain.
6. Runpod cost/control. Remote compute must be bounded and approved.
7. Public branch vs local OpenClaw branch. The GitHub branch has the rich historical harness; the OpenClaw local branch has the newer SLT-validation scaffold and HDPC worktree setup.

Verification performed while preparing this handoff

Commands run locally in /home/openclaw/research-agent or subdirectories:

```
gh repo view bgoertzel-sing/relaleap --json
url,defaultBranchRef,isPrivate,licenseInfo,pushedAt
git ls-remote https://github.com/bgoertzel-sing/relaleap.git 'refs/heads/*'
git clone https://github.com/bgoertzel-sing/relaleap.git scratch/relaleap-gh-inspect
git -C scratch/relaleap-gh-inspect log --oneline --decorate --all --max-count=40
python3 -m pytest tests -q  # in projects/relaleap/repos/relaleap
```

Observed verification results:

- GitHub repo public at https://github.com/bgoertzel-sing/relaleap
- GitHub branch agent/slt-pregate-v2-cache present at 5b959e3...
- OpenClaw local tests on projects/relaleap/repos/relaleap: 93 passed in 17.07s
- No Runpod provisioning or paid compute action was taken.

Bottom line

RelaLeap has a useful, artifact-oriented residual-layer experimentation framework and a long trail of conservative negative and partially positive results. The best current use of that history is not to keep iterating the old top-k/ACSR/PC-proxy branches. It is to reuse the discipline: small model, identity invariants, artifact contracts, matched controls, and fail-closed decision reports.

The next clean attack is HDPC/ePC on Tiny Shakespeare: first prove the predictive-coding wrapper and homotopy path on a tiny model, then add a PC crown, and only after that consider SLT inputs and columnar residual structure as upgrades.